

IA Aplicada com LLMs

Aula 03 — Tool calling: quando o prompt vira schema

Onde paramos

A Aula 02 terminou uma seção com uma frase pedindo para ser guardada:

"Quando o modelo 'chama uma função', ele está gerando um JSON restrito pelo schema dos parâmetros dessa função."

Este deck paga essa promessa.

E fecha o arco da aula

Começamos com prompt como **texto livre** — instruções em português, exemplos, "pense passo a passo".

Terminamos com o prompt virando **contrato tipado**, verificável por máquina.

Enquanto a saída é texto	Quando é chamada validada
o seu programa interpreta	o seu programa executa

E é o que o nome do curso promete

Até aqui vocês construíram programas que **leem, processam e geram**.

A partir daqui:

decidem e agem

Roteiro

1. **É o mesmo mecanismo** da saída estruturada
2. **Os quatro tempos**
3. **Declarar a ferramenta**
4. **A descrição é prompt**
5. **O laço**
6. **ReAct** — o nome do que vocês acabaram de escrever
7. **Projetar ferramenta** é o trabalho de verdade

É o mesmo mecanismo — literalmente

```
# Aula 02 – forçar a saída num formato
response_format={"type": "json_schema",
                 "json_schema": {"name": "cadastro", "schema": SCHEMA}}
```

```
# Aula 03 – oferecer uma ferramenta
tools=[{"type": "function",
        "function": {"name": "consultar_pedido",
                     "description": "...",
                     "parameters": SCHEMA}}] # ← o MESMO tipo de objeto
```

`parameters` é um **JSON Schema**, com a mesma decodificação restrita.

A diferença não é o mecanismo

	Saída estruturada	Tool calling
O modelo produz	um JSON com os dados	JSON com função + argumentos
O seu código	grava, exhibe, valida	executa a função
<code>finish_reason</code>	<code>"stop"</code>	<code>"tool_calls"</code>

Sem saída estruturada confiável não existe agente confiável.

Agora dá para ver por quê: se o formato não fosse garantido, você estaria executando funções a partir de **texto adivinhado**.

Os quatro tempos

1. VOCÊ declara `tools=[...]` junto com as mensagens
2. O MODELO decide `finish_reason="tool_calls"` + nome e argumentos
3. O SEU CÓDIGO executa a função Python, pega o retorno
4. VOCÊ devolve `role="tool"` e chama o modelo DE NOVO
volta ao 2, até responder sem pedir ferramenta

Quem faz o quê

Dois tempos são **seus** (1 e 3–4). Um é do modelo (2).

O modelo decide. O seu código age.

Ele **nunca executa nada** — nem sabe que as suas funções existem como código. Só viu uma **descrição** delas.

Toda a segurança do sistema mora nessa separação.

Declarando a ferramenta

```
{
  "type": "function",
  "function": {
    "name": "consultar_pedido",
    "description": "Consulta a situação e a previsão de entrega...",
    "parameters": {
      "type": "object",
      "properties": {
        "numero": {"type": "string", "pattern": "^[0-9]+$",
          "description": "Número do pedido, somente dígitos."},
      },
      "required": ["numero"],
      "additionalProperties": false,
    }
  }
}
```

`pattern` e `additionalProperties` são as mesmas garantias da Aula 02.

A descrição da ferramenta É prompt

O modelo **não vê o seu código.**

Tudo o que ele tem para decidir **se e quando** chamar:

1. o **nome** da função
2. a **descrição**
3. a descrição de cada **parâmetro**

Logo: vale tudo o que o deck 01 ensinou. **Cada frase elimina uma possibilidade.**

Três descrições

Descrição	Problema
"Consulta pedido"	não diz quando usar
"Consulta de forma eficiente e confiável..."	adjetivos não restringem nada
"Consulta a situação... Use quando o cliente citar um número de pedido e a resposta depender do status real. **Não use para dúvidas gerais.**"	

O "*não use para...*" é o **exemplo negativo** do deck 01, aplicado a ferramenta.

Sem ele: o agente consulta o banco para responder "*vocês entregam no sábado?*".

A frase que resume a aula

O prompt não desapareceu — ele foi para dentro do schema.

Continua sendo texto em português que decide comportamento.

Só que agora **num campo com contrato**.

Lendo a chamada

```
msg = resposta.choices[0].message

if msg.tool_calls:
    mensagens.append(msg)                    # 1x: a fala do assistente

    for chamada in msg.tool_calls:          # 1x por chamada
        nome = chamada.function.name
        argumentos = json.loads(chamada.function.arguments)
        resultado = FERRAMENTAS[nome](**argumentos)

        mensagens.append({
            "role": "tool",
            "tool_call_id": chamada.id,      # amarra resultado à chamada
            "name": nome,
            "content": json.dumps(resultado, ensure_ascii=False),
        })
```

Quatro armadilhas da primeira implementação

1. `msg` **uma vez**, mensagem `tool` **uma por chamada** — trocar a ordem costuma render 400 do provedor
2. `tool_call_id` **é obrigatório** — amarra cada resultado à sua chamada
3. `content` **é string** — serialize com `json.dumps`
4. `temperature=0` — escolher ferramenta é a decisão mais crítica: erro aqui não gera texto ruim, gera **ação errada no mundo**

O laço

```
for passo in range(max_passos):  
    r = client.chat.completions.create(  
        model=MODELO, messages=mensagens,  
        tools=DECLARACOES, temperature=0)  
    msg = r.choices[0].message  
  
    if not msg.tool_calls:                # respondeu: terminou  
        return msg.content  
  
    mensagens.append(msg)  
    for chamada in msg.tool_calls:  
        ...                               # executa e devolve  
  
raise RuntimeError(f"não concluiu em {max_passos} passos")
```


Por que o teto de passos não é paranoia

Modelos **entram em laço**: chamam a mesma ferramenta repetidamente, ou alternam entre duas sem convergir.

Sem teto, isso é **uma conta aberta rodando sozinha**.

E lembre: **cada volta reenvia todo o histórico** — o contexto vai inchando a cada passo, com o raciocínio e os resultados anteriores.

Num agente, a janela é o recurso que mais rápido se esgota.

Este laço tem nome

ReAct

Reasoning + Acting (Yao et al., 2022)

Vocês já o encontraram na **Aula 01, nota 03**, como o padrão de agente mais simples e mais usado.

Agora acabaram de **escrevê-lo**.

A correspondência é direta

No paper	No seu código
Thought	o modelo decide → <code>finish_reason="tool_calls"</code>
Action	<code>FERRAMENTAS[nome](**argumentos)</code>
Observation	a mensagem <code>role: "tool"</code> com o resultado
o laço	<code>for passo in range(max_passos)</code>

A ideia do paper: **raciocinar e agir se ajudam mutuamente.**

O raciocínio decide a ação; a observação corrige o raciocínio seguinte.

Como era antes do tool calling

ReAct é de **2022** — não havia campo `tools` na API.

O laço era emulado **no texto**:

```
Thought: preciso saber o status do pedido  
Action: consultar_pedido[48219]  
Observation: em transporte, previsão 02/09  
Thought: já sei a resposta  
Final Answer: ...
```

E o programa procurava a linha `Action:` com uma **expressão regular**.

E quebrava do jeito que a Aula 02 previu

- `Action : consultar_pedido` — um espaço a mais
- `Ação:` — em português
- o modelo explicando o que ia fazer **antes** da linha
- os colchetes esquecidos

Cada variação quebrava o regex. A correção era **escrever um prompt melhor, e torcer.**

O que mudou

O `tools` faz a mesma coisa, com **schema em vez de texto**.

Em vez de **pedir** que o modelo escreva `Action:` no formato certo, a decodificação restrita **impede** que ele escreva outra coisa.

O ReAct não mudou.

A estrutura saiu do prompt e foi para o schema.

⚠ Se um tutorial manda escrever `Thought/Action/Observation` num modelo com tool calling, é material anterior a 2023 — ou cópia dele.

Projetar a ferramenta é o trabalho de verdade

Critério	Regra
Nome	verbo + objeto: <code>consultar_pedido</code> , não <code>get_data</code>
Granularidade	poucas boas > 40 mal definidas
Descrição	o que faz, quando usar , quando não
Erros legíveis	devolva <code>{"erro": ...}</code> , nunca levante exceção
Retorno enxuto	200 registros = 8 mil tokens em toda volta
Leitura × escrita	escrever exige confirmação, teto e registro

O item mais contraintuitivo

Em programação comum, exceção é o certo: **falhe cedo e ruidosamente**.

Aqui, a exceção **mata o laço**.

```
return {"erro": f"pedido {numero} não encontrado",  
        "dica": "confirme o número com o cliente"}
```

Erro devolvido como texto vira **informação no contexto** — e o modelo se recupera sozinho: pede o número de novo, tenta outra ferramenta, explica.

A ferramenta conversa com um **leitor de texto**, não com um `try/except`.

O modelo não ficou mais inteligente

```
[ferramenta] consultar_pedido({'numero': '48219'})  
-> {"situacao": "em transporte", "previsao": "2026-09-02"}
```

Ele continua **sem saber nada** sobre os seus pedidos.

O que mudou: agora existe **um caminho** para a informação chegar, e ele reconheceu que devia usá-lo.

"Pare de tentar fazer o LLM saber coisas. Faça-o decidir coisas."

— Aula 01, nota 03

O que fica para a próxima aula

Vocês têm **uma volta** funcionando. Falta o que vira sistema:

- **estado e memória** entre execuções
- **múltiplas ferramentas** e a escolha entre elas
- **outros padrões**: planejamento, reflexão, roteador, humano no laço
(o ReAct vocês já têm)
- **MCP** — conectar a ferramentas de terceiros sem escrever cada integração
- **confiabilidade**: retry, detecção de laço, orçamento, checkpoint
- **context engineering dinâmica** — o deck 02, agora ao longo da trajetória

Mas o **mecanismo** vocês já viram inteiro.

Síntese

- **Tool calling é saída estruturada.** `parameters` é um JSON Schema; a diferença é que você **executa** em vez de gravar
- **Quatro tempos:** você declara → **o modelo decide** → **seu código executa** → você devolve `role: "tool"` e chama de novo
- **O modelo nunca executa nada.** Toda a segurança mora nessa separação
- **A descrição da ferramenta é prompt** — diga **quando usar** e **quando não**
- `tool_call_id` obrigatório · `content` é string · **`temperature=0`**
- **Teto de passos** é obrigatório: cada volta reenvia todo o histórico
- **Erro se devolve, não se levanta** — e escreva a mensagem para ser lida
- **Este laço é o ReAct:** *Thought* = a decisão do modelo, *Action* = o seu código executando, *Observation* = a mensagem `role: "tool"`
- Antes do tool calling isso era emulado **em texto** e quebrava no regex.